

PetSitterHub — Guida al deploy

Come mettere online il progetto su Render (piattaforma cloud). Configurazione, ambienti e rollback.

Panoramica

Il progetto è composto da tre parti online: un database PostgreSQL, il backend (Node/Express) e il frontend (React). Sono tutti ospitati su Render. Ogni push su GitHub aggiorna automaticamente backend e frontend (deploy continuo).

Prerequisiti

- Un account Render collegato al repository GitHub del progetto.
- Le chiavi dei servizi esterni: Stripe (pagamenti), Cloudinary (foto), account email.
- psql installato sul proprio PC per caricare il database.

1. Database

Su Render: New + → Postgres. Nome petsitterhub-db, region Frankfurt, piano Free. Attendere lo stato Available.

Caricare le tabelle e i dati dal proprio PC usando la External Database URL (sezione Connections):

```
psql "EXTERNAL_URL" -f database/01-schema.sql
psql "EXTERNAL_URL" -f database/02-seed.sql
```

Annotare la Internal Database URL: servirà al backend.

2. Backend

Su Render: New + → Web Service, collegato al repository. Impostazioni:

Campo	Valore
Root Directory	backend
Build Command	npm install
Start Command	npm start
Piano	Free

Variabili d'ambiente (Environment) da impostare:

Variabile	Contenuto
DATABASE_URL	Internal Database URL del passo 1
JWT_SECRET	Stringa lunga e casuale per firmare i token
STRIPE_SECRET_KEY	Chiave segreta Stripe (test)
CLOUDINARY_CLOUD_NAME	Nome cloud di Cloudinary
CLOUDINARY_API_KEY	API key di Cloudinary
CLOUDINARY_API_SECRET	API secret di Cloudinary
FRONTEND_URL	Indirizzo del frontend (dal passo 3)

Al termine, annotare l'URL pubblico del backend.

3. Frontend

Su Render: New + → Static Site, stesso repository. Impostazioni:

Campo	Valore
Root Directory	frontend
Build Command	npm install && npm run build
Publish Directory	dist

Variabile d'ambiente:

VITE_API_URL = <URL del backend>/api

Importante: le variabili VITE_ vengono incorporate durante la build. Se si modifica VITE_API_URL dopo, occorre rifare il deploy con 'Clear build cache & deploy'.

4. Collegamento finale

Tornare al backend → Environment → impostare FRONTEND_URL con l'indirizzo del frontend appena creato. Questo permette al backend di accettare le richieste del sito (sicurezza CORS).

5. Verifica

Aprire <URL backend>/api/health: deve rispondere che il database è connesso. Poi aprire il sito e provare login, catalogo e una prenotazione. Nota: sul piano gratuito il primo accesso dopo un periodo di inattività richiede 30-60 secondi di risveglio.

Aggiornare il progetto

Il codice si aggiorna con un semplice push su GitHub: Render ricostruisce backend e frontend da solo. Le modifiche allo schema del database, invece, non sono automatiche: vanno applicate a mano con psql sul database cloud (es. ALTER TABLE per nuove colonne).

Gestione errori

Sintomo	Causa probabile e soluzione
Sito non raggiungibile	Servizio in pausa (piano free): attendere il risveglio e ricaricare.
database non raggiungibile	DATABASE_URL errata: usare la Internal URL nel backend.
Il sito non mostra dati	VITE_API_URL mancante/errata nel frontend, oppure build da rifare.
password authentication failed	Credenziali database non corrispondenti tra .env e database.
Invalid API Key (Stripe/Cloudinary)	Chiave mancante o errata nelle Environment del backend.

Rollback (tornare a una versione precedente)

Se un aggiornamento introduce un problema, si può tornare a una versione funzionante in due modi:

- Da Render: nel servizio, tab 'Events' o 'Deploys', scegliere un deploy precedente andato a buon fine e selezionare 'Rollback' / 'Redeploy'.
- Da Git: annullare l'ultimo commit problematico e ripushare:

```
git revert HEAD
git push
```

Render ridispegnerà automaticamente la versione corretta. Per le modifiche al database, tenere sempre uno script inverso o un backup prima di cambiamenti importanti.